

THE UNIVERSITY OF AZAD JAMMU & KASHMIR, MUZAFFARABAD

Department of Software Engineering

LAB 04

Submitted to: Engr. Dr. Asma Javaid

Session: 2024-28

Roll No: 2024-SE-34

Date: 27-10-2026

Jawahir Ali

1. Online Doctor Appointment Management System

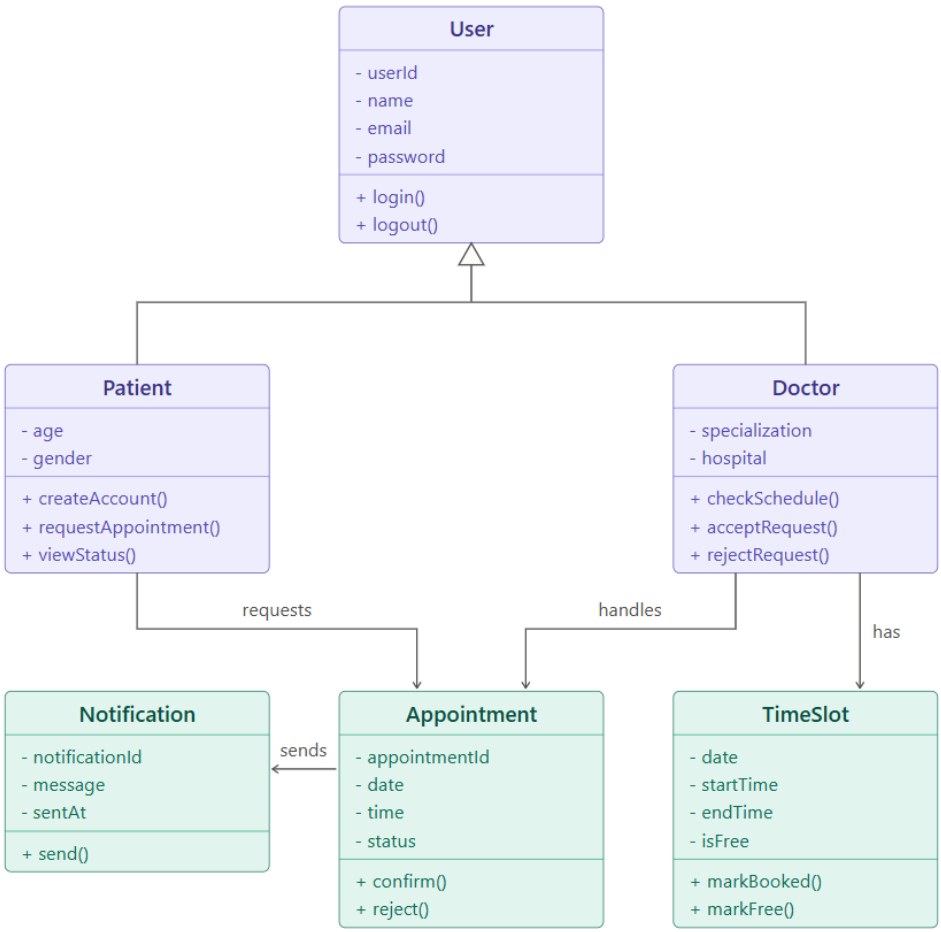


Figure 1. Online Doctor Appointment Management System

Purpose:

The system lets patients create accounts and request appointments with specialists. Doctors have their own accounts, see the incoming requests, and accept or reject them depending on whether they have a free slot. Accepted requests send the patient a confirmation.

Classes and their roles

1. **User** is the general class for anyone who logs in. It holds the common data *(userId, name, email, password)* and the common operations *(login, logout).* It exists so that patients and doctors do not repeat the same code.

2. **Patient** is a type of User who needs medical care. It adds age and gender, and it can create an account, request an appointment, and view the status of that request.
3. **Doctor** is a type of User who provides care. It adds specialization and hospital/clinic. A doctor can check his or her schedule and accept or reject appointment requests.
4. **Appointment** represents one request from a patient to a doctor. It stores the appointment ID, date, time, and status (Pending, Accepted, or Rejected), and it can be confirmed or rejected.
5. **TimeSlot** represents one period in a doctor's schedule. It stores the date, start time, end time, and whether the slot is free. It is what decides whether a request can be accepted.
6. **Notification** is the confirmation message sent to the patient. It stores the message and the time it was sent, and it has a send() operation

Relationships

- Patient and Doctor inherit from User (an "is-a" relationship).
- A patient can make many appointment requests, and a doctor handles many of them.
- A doctor has many time slots.
- Each accepted appointment generates a notification.

How it works:

The patient logs in and requests an appointment. The doctor checks his or her TimeSlot list. If a free slot exists, the doctor accepts, the slot is marked as booked, the appointment status becomes Accepted, and a Notification is sent to the patient. If no slot is free, the doctor rejects the request and the status becomes Rejected.

2. Animal, Cat and Dog

Purpose:

This task demonstrates inheritance. Animal is the superclass with the behavior common to all animals, and Cat and Dog are subclasses that reuse it and add their own behavior.

- Animal has a protected attribute name and two public functions, eat() and sleep(). The attribute is protected so the subclasses can use it while outside code cannot.
- Cat inherits from Animal using public inheritance, so a cat can eat and sleep too. It adds meow().
- Dog inherits in the same way and adds bark().
- Each subclass constructor passes the name to the Animal constructor using an initializer list (Animal(n)).

In main(), a Cat and a Dog are created. Both can call eat() and sleep(), which come from Animal, and each calls its own special function (meow() or bark()). This shows code reuse: eating and sleeping are written once, not twice.

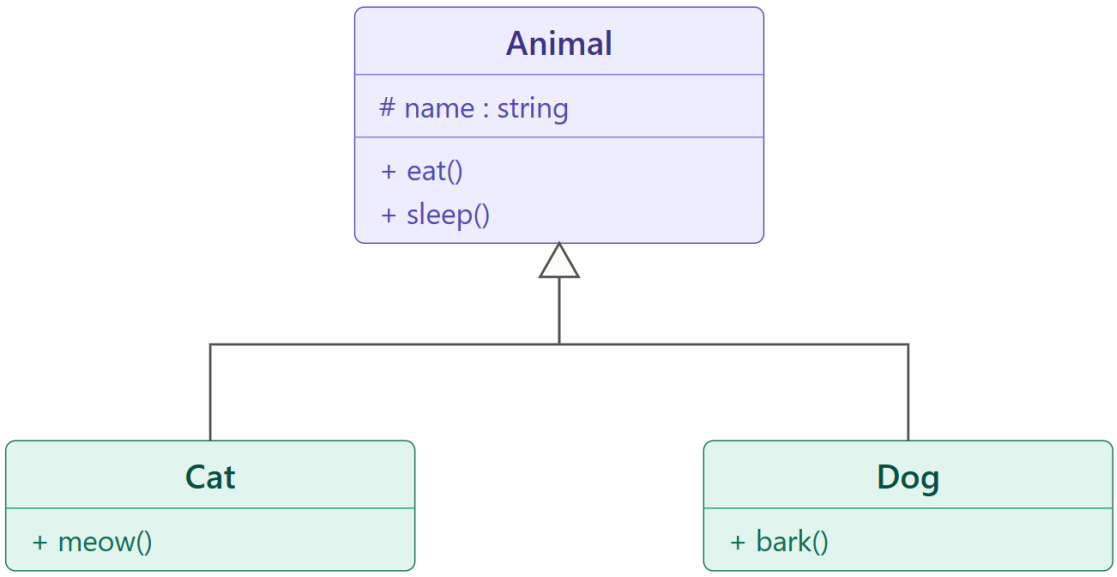


Figure 2. Animal, Cat and Dog

C++ Code:

```
#include <iostream>
#include <string>
using namespace std;

// Superclass
class Animal {
protected:
    string name;
public:
    Animal(string n) : name(n) {}

    void eat() {
        cout << name << " is eating." << endl;
    }

    void sleep() {
        cout << name << " is sleeping." << endl;
    }
};

// Subclass 1
class Cat : public Animal {
public:
    Cat(string n) : Animal(n) {}
```

```
    void meow() {
        cout << name << " says: Meow!" << endl;
    }
};

// Subclass 2
class Dog : public Animal {
public:
    Dog(string n) : Animal(n) {}

    void bark() {
        cout << name << " says: Woof! Woof!" << endl;
    }
};
int main() {
    Cat c("Kitty");
    Dog d("Tommy");
    cout << "--- Cat ---" << endl;
    c.eat();
    c.sleep();
    c.meow();
    cout << "--- Dog ---" << endl;
    d.eat();
    d.sleep();
    d.bark();
    return 0;}
```

3. Shape, Triangle and Rectangle

Purpose:

This task demonstrates inheritance together with polymorphism. Different shapes calculate their area in different ways, but they all follow the same interface.

- **Shape** is an abstract superclass. It has a protected attribute area and two pure virtual functions, getInput() and calculateArea(), which have no body (= 0). This forces every subclass to implement them. It also has a normal function displayArea() that prints the result, so that code is written only once. The virtual destructor makes sure objects are deleted correctly when they are used through a base-class pointer.
- **Triangle** has the attributes base and height. Its getInput() reads them from the user, and its calculateArea() computes ***½ × base × height.***
- **Rectangle** has the attributes length and width. Its getInput() reads them from the user, and its calculateArea() computes **length × width.**

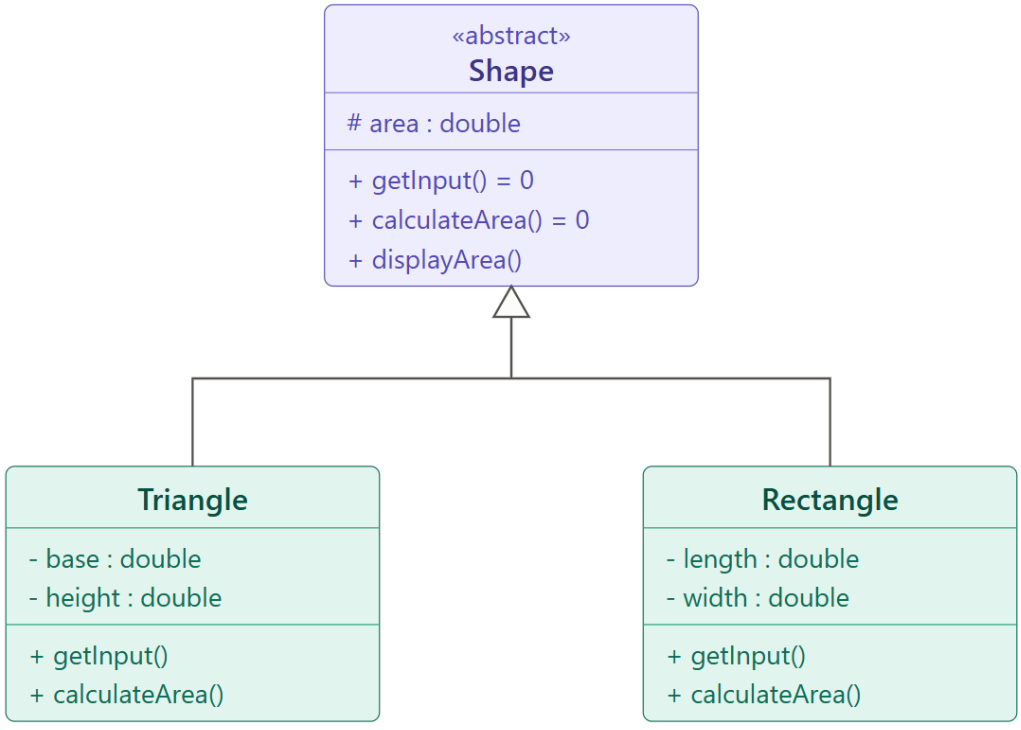


Figure 3. Shape, Triangle and Rectangle

In main(), the addresses of a Triangle and a Rectangle are stored in an array of **Shape* pointers**. A loop calls getInput(), calculateArea() and displayArea() on each one. Because the functions are virtual, C++ picks the correct subclass version at run time. This is polymorphism: one loop and one interface work for every kind of shape, and a new shape (such as a Circle) could be added without changing the loop.

C++ Code:

```
#include <iostream>

using namespace std;

class Shape {

protected:

   double area;

public:

   Shape() : area(0) {}

   virtual void getInput() = 0;

   virtual void calculateArea() = 0;

   void displayArea() { cout << "Area = " << area << endl; }

   virtual ~Shape() {}

};

class Triangle : public Shape {

   double base, height;

public:

   void getInput() override {

      cout << "Enter base and height of the triangle: ";

      cin >> base >> height;

   }

   void calculateArea() override {
```

```
        area = 0.5 * base * height;
    }
};
class Rectangle : public Shape {
    double length, width;
public:
    void getInput() override {
        cout << "Enter length and width of the rectangle: ";
        cin >> length >> width;
    }
    void calculateArea() override {
        area = length * width;
    }
};
int main() {
    Triangle t;
    Rectangle r;
    Shape* shapes[2] = { &t, &r };

    for (int i = 0; i < 2; i++) {
        shapes[i]->getInput();
        shapes[i]->calculateArea();
        shapes[i]->displayArea();
    }
    return 0;
}
```